

Homigo Care Admin Panel Blueprint

- [HOMIGO CARE — ADMIN PANEL](#)
 - [Complete Build Blueprint for an AI Coding Agent \(Manus AI\)](#)
 - [Repository, Infrastructure, Real Cross-Platform Connection, Design System, Screen-by-Screen Specification](#)
 - [0. Purpose, Working Method & When to Ask Questions](#)
 - [1. Repository & Infrastructure Setup](#)
 - [1.1 GitHub Repository](#)
 - [1.2 Vercel \(Host — via Vercel MCP\)](#)
 - [1.3 Firebase — Same Project, Admin-Level Reads](#)
 - [1.4 Supabase — Same Project, Elevated Access](#)
 - [1.5 Cloudinary — Same Account, Admin-Level Access](#)
 - [1.6 EmailJS — Owned Entirely by This Repo](#)
 - [1.7 Environment Variables](#)
 - [1.8 AI Image-Generation Request Pipeline — 01_ai_images.json](#)
 - [1.9 Verifying Real Cross-Platform Connection \(do this before building any feature screen\)](#)
 - [2. Design System — Consistent With the Three Client Apps, Adapted for Dense Data Work](#)
 - [3. Technology Stack \(Admin Panel — final, authoritative\)](#)
 - [4. Icons & Visual Assets — Admin-Specific Decision Rule](#)
 - [5. Access Control — 5 Admin Roles \(authoritative, enforced at the database layer\)](#)
 - [6. Total Screen Count](#)
 - [7. Complete Screen-by-Screen Blueprint](#)
 - [7.1 Authentication \(1 screen\)](#)
 - [7.2 Dashboard Shell & Overview \(1 screen\)](#)
 - [7.3 Content Management System — CMS \(7 screens\)](#)
 - [7.4 Verification & Assessment \(3 screens\)](#)
 - [7.5 Bookings, Live Map & Emergency \(3 screens\)](#)
 - [7.6 Finance — Manual Payment System \(3 screens\)](#)
 - [7.7 Reviews Moderation \(1 screen\)](#)
 - [7.8 Disputes \(1 screen, shared with Support Admin\)](#)
 - [7.9 Analytics \(1 screen\)](#)
 - [7.10 Admin User Management & Notifications \(2 screens\)](#)
 - [8. Navigation Map](#)
 - [9. Cross-Platform Data Sync — The Complete Picture](#)
 - [10. Build & Deployment Pipeline](#)
 - [11. Notes for the AI Developer \(Guide, Not a Separate File Yet\)](#)
 - [12. Final QA Checklist Before Calling the Build “Done”](#)

HOMIGO CARE — ADMIN PANEL

Complete Build Blueprint for an AI Coding Agent (Manus AI)

Repository, Infrastructure, Real Cross-Platform Connection, Design System, Screen-by-Screen Specification

0. Purpose, Working Method & When to Ask Questions

This is the **fourth and final repository** in the Homigo Care system — the **command center** that the other three (HOMIGO-CARE-ANDROID, HOMIGO-CARE-IOS, HOMIGO-CARE-WEB) all report into and are controlled from. It is web-only, hosted on Vercel, and its entire reason for existing is to give the founder **real, live, full CRUD control** over every piece of content, every verification decision, every manual payment, and every emergency alert flowing through all three client apps — with **zero fake data anywhere**.

MCP servers available to the agent: GitHub, Supabase, Firebase CLI, Vercel, Cloudinary — the same five, and this repo is where **all of them come together**, since the Admin Panel is the one project that legitimately needs elevated access to every service at once.

Working method — build in phases: 1. **Phase 0** — repository + infrastructure wiring (\$1), confirm live connectivity to the *same* Firebase project, Supabase database, and Cloudinary account already used by the three client apps (\$1.9) — do not create new instances of any of these. 2. **Phase 1** — Auth + 5-role access control (\$5) + the shared design system/layout shell (\$2) with an empty dashboard. 3. **Phase 2** — CMS modules (\$7.3): FAQ, Support Center, Legal, About, Pricing, Role Requirements — these unblock the website’s public pages and the apps’ dynamic registration engine, so build them early. 4. **Phase 3** — Verification & Assessment modules (\$7.4) — unblocks real professionals going live. 5. **Phase 4** — Bookings, Live Map, Emergency/SOS modules (\$7.5) — unblocks real bookings being visible. 6. **Phase 5** — Finance/Manual Payments module (\$7.6) — unblocks real money flow. 7. **Phase 6** — Reviews Moderation, Analytics, Admin User Management, Notifications Composer (\$7.7–\$7.10) — completes the system.

Ask, and keep working. If a required credential, a business-rule decision (e.g. exact commission edge case), or a real asset (admin dashboard illustrations, the founder’s own admin account details) isn’t available yet, ask explicitly and continue building whatever doesn’t depend on it — never block the whole panel on one missing piece.

1. Repository & Infrastructure Setup

1.1 GitHub Repository

- Using the **GitHub MCP**, create (or, if one exists, confirm with the founder before reusing/resetting — same pattern as the other three repos) a repository named exactly HOMIGO-CARE-ADMIN.
- Folder structure:

```
HOMIGO-CARE-ADMIN/
├── app/
│   ├── (auth)/login/                # Separate admin login, never shared with patient/professional login
│   ├── (dashboard)/
│   │   ├── overview/                # A-02
│   │   ├── verification/            # A-03, A-04
│   │   ├── assessments/             # A-05
│   │   ├── bookings/                # A-06, A-07
│   │   ├── emergency/               # A-08
│   │   ├── disputes/                # A-09
│   │   ├── reviews/                # A-10
│   │   ├── finance/
│   │   │   ├── payment-accounts/    # A-11
│   │   │   ├── verification-queue/  # A-12
│   │   │   └── payouts/              # A-13
│   └── cms/
│       ├── faq/                     to A-20
│       ├── support/
│       ├── legal/
│       ├── about/
│       ├── announcements/
│       ├── pricing/
│       ├── role-requirements/       # A-14
│       ├── analytics/               # A-21
│       ├── admins/                  # A-22
│       ├── notifications-composer/  # A-23
│       └── layout.tsx               # Sidebar shell, role-aware nav
├── components/ui/                   # shadcn/ui, Tailwind-themed to brand tokens
├── lib/
│   ├── firebase/                   # Admin-side Firestore listeners (live map, live bookings)
│   ├── supabase/                   # Supabase JS client, service-role usage confined to Server Actions only
│   ├── cloudinary/                 # Signed URL generation for document review
│   └── emailjs/                    # EmailJS SMTP trigger logic (this repo OWNS the EmailJS credentials)
├── app/api/                         # Public-facing API routes the three client apps call:
│   ├── otp/send/route.ts
│   ├── otp/verify/route.ts
│   ├── cms/[key]/route.ts
│   └── revalidate/route.ts
├── public/assets_pending/
│   └── 01_ai_images.json           # Admin-only dashboard iconography requests – see §4
├── docs/
│   └── this-blueprint.pdf
└── README.md
```

- Critical architectural fact for the agent to internalize:** the app/api/otp/* and app/api/cms/* routes living in **this** repository are the exact same endpoints the Android Blueprint (§1.5), the iOS Blueprint (§1.5), and the Web Blueprint (§1.8) documents told those three repos to call at ADMIN_API_BASE_URL. This Admin Panel is not just a viewer — it is the **backend-for-frontend** that all three client apps quietly depend on for OTP delivery and CMS content. Building this repo correctly is what makes the other three actually work end-to-end, not just look right in isolation.

1.2 Vercel (Host — via Vercel MCP)

- Create a Vercel project linked to HOMIGO-CARE-ADMIN, Next.js preset, custom domain admin.homigocare.pk once available.
- This is the **only** repo that should hold the EmailJS SMTP credentials, the Cloudinary API secret, and the Supabase **service-role** key (elevated, bypasses RLS) — all as Vercel encrypted Environment Variables, never exposed to any client bundle, and only ever used inside Server Actions/Route Handlers, never in a "use client" component.

1.3 Firebase — Same Project, Admin-Level Reads

Connect to the **exact same** homigo-care-prod (and -dev) Firebase project already created for the Android app. Do not create a new project. Enable the Admin Panel to read (not write, except where explicitly needed, e.g. sending a targeted push) from: presence, bookings_live, chat_threads (for dispute evidence), notifications_feed. For sending **admin-triggered push notifications** (the Notifications Composer, §7.10), use the **Firebase Admin SDK** server-side (Node.js), authenticated with a service-account JSON key stored as a Vercel secret — this is different from the client SDKs the three apps use, and is appropriate specifically because this repo is the trusted backend.

1.4 Supabase — Same Project, Elevated Access

Connect to the **exact same** homigo-care Supabase project. This repo is the one place allowed to use the **service-role key**

(server-side only, inside Server Actions) to perform actions Row Level Security would otherwise block for a normal user — e.g. an admin approving another admin’s account, or force-editing a booking during dispute resolution. All admin-initiated reads/writes should still prefer going through the admin-role JWT + RLS policies (§5) wherever that’s sufficient, reserving the service-role key for the few genuinely cross-cutting actions RLS can’t express.

1.5 Cloudinary — Same Account, Admin-Level Access

Connect to the same Cloudinary account/folders as the three client apps (§1.4 of the Android Blueprint). This repo additionally holds the **API secret** needed to generate signed viewing URLs for CNIC/degree/prescription/receipt documents on demand (the Verification Queue and Payment Verification Queue screens depend entirely on this) and to generate signatures for the homigo_user_documents signed preset that the client apps request before uploading.

1.6 EmailJS — Owned Entirely by This Repo

Set up the EmailJS account/service/template here. This repo’s app/api/otp/send/route.ts is the **only** place in the entire Homigo Care system that calls EmailJS. Store the EmailJS Service ID, Template ID, and Public/Private keys as Vercel secrets. Templates needed: OTP code, password-reset code, document-approved notice, document-rejected notice (with reason), booking-receipt copy, weekly professional earnings summary, admin-login-from-new-device alert.

1.7 Environment Variables

```
NEXT_PUBLIC_FIREBASE_API_KEY=...           # same project as the three apps
FIREBASE_ADMIN_SERVICE_ACCOUNT_JSON=...     # server-side only, for Admin SDK push sends
NEXT_PUBLIC_SUPABASE_URL=...
NEXT_PUBLIC_SUPABASE_ANON_KEY=...
SUPABASE_SERVICE_ROLE_KEY=...               # server-side only, never exposed to the browser
CLOUDINARY_CLOUD_NAME=...
CLOUDINARY_API_KEY=...
CLOUDINARY_API_SECRET=...                  # server-side only
EMAILJS_SERVICE_ID=...
EMAILJS_TEMPLATE_ID_OTP=...
EMAILJS_TEMPLATE_ID_DOC_STATUS=...
EMAILJS_PRIVATE_KEY=...                     # server-side only
```

1.8 AI Image-Generation Request Pipeline — 01_ai_images.json

Same schema and 20–50-item hard limits as the other three repos. The Admin Panel’s own visual needs are narrower (it’s an internal tool, not a consumer-facing brand surface) — typically one batch covering: dashboard module icons if Lucide doesn’t have a close enough match, an empty-state illustration set for each module’s “no data yet” screen (Verification Queue empty, Bookings empty, Disputes empty, Finance queue empty), and a simple “Admin” avatar placeholder set per the 5 roles. Living at public/assets_pending/01_ai_images.json in this repo, same rules: exact subject, background type, brand-locked flat-vector style, exact destination path.

1.9 Verifying Real Cross-Platform Connection (do this before building any feature screen)

Before writing a single dashboard screen, the agent must **prove** the four repositories are genuinely wired together, not just individually functional: 1. Create one real test patient account through the actual Android app (or iOS, or Web — whichever is furthest along) → confirm the resulting row appears in the **same** Supabase users/patients tables this Admin Panel queries. 2. Approve a real test professional’s documents from this Admin Panel’s Verification Queue → confirm the professional’s app (Android/iOS/Web, whichever they registered on) receives the real-time status change (via the shared Firestore booking_status_events/a dedicated verification_status document, or a Supabase real-time subscription) without needing to force-refresh. 3. Trigger one push notification from the Notifications Composer → confirm it actually arrives on a real device/browser for each of the three client platforms. 4. Only once this round-trip is confirmed working end-to-end should the agent consider the infrastructure phase (Phase 0) complete and move on to full feature-screen development.

2. Design System — Consistent With the Three Client Apps, Adapted for Dense Data Work

Use the **exact same brand tokens** as the Android/iOS/Web blueprints (primary teal #1B6B63, dark teal #0D4A44, header emerald #0B5545, orange #F5823A, error red #E53935, success green #43A047, gold/silver badge accents) — a founder glancing at the Admin Panel right after the app should feel zero brand disconnect. The difference here is **density and information hierarchy**, not color:

- **Layout philosophy:** unlike the consumer apps (generous whitespace, large touch targets), the Admin Panel favors **data-dense tables, compact rows, and sidebar navigation** — this is a professional back-office tool used on a desktop/laptop, not a consumer mobile screen.
- **Cards:** same 15px radius, but smaller padding (12px instead of 16–24px) so more information fits per screen.
- **Buttons/inputs:** same 25px pill radius for primary actions (Approve, Confirm Paid, Send), but data tables use standard rectangular cells, not pill-shaped rows.
- **Status colors (consistent everywhere in this panel):** Pending = MediumGray badge, Approved/Paid/Confirmed = SuccessGreen badge, Rejected/Failed = ErrorRed badge, In Review = SecondaryOrange badge, Elite/Priority = GoldElite

badge.

- **Typography:** same Plus Jakarta Sans family as the website, slightly smaller base sizes for table density (Body 13px instead of 14px is acceptable here specifically, given the professional/desktop context — this is the one intentional deviation from the consumer apps’ exact type scale, and should be defined once in this repo’s own `tailwind.config.ts`, not silently improvised per-component).
- **Light mode only**, matching every other Homigo Care surface.

3. Technology Stack (Admin Panel — final, authoritative)

Layer	Choice	Reason
Framework	Next.js 14+ (App Router) , TypeScript	Same framework family as the public website, natural Vercel deployment, Server Actions ideal for the secret-holding operations this repo uniquely needs
Styling	Tailwind CSS + shadcn/ui (data tables, dialogs, dropdowns, tabs, toasts)	Accessible, fast to build a data-heavy back-office UI with, restyleable to brand tokens
Tables	TanStack Table (headless, pairs with shadcn’s table styling)	Sorting/filtering/pagination for Bookings, Verification Queue, Finance Queue, Disputes — all inherently tabular
Charts	Recharts	Same library as the website’s Vitals graphs — Analytics module reuses the same charting language
Auth	Supabase Auth , dedicated admin role, TOTP 2FA (Supabase Auth’s MFA support)	Separate identity space from patient/professional Firebase Auth accounts — an admin login must never be confusable with a consumer account
Realtime	Firebase Firestore SDK (read-heavy: live map, live bookings, live chat evidence)	Same real-time backend the client apps use
Structured data	Supabase JS client (anon key for RLS-scoped reads, service-role key confined to Server Actions for the few elevated operations)	One shared Postgres database across the whole system
Maps	Leaflet + OpenStreetMap (react-leaflet) for the Live Bookings Map	Free, matches the client apps’ default maps provider
File access	Cloudinary Admin API (server-side signed URL generation)	Secure, time-limited viewing of sensitive documents
Email	EmailJS (server-side trigger only, via <code>@emailjs/nodejs</code> or a plain REST call from a Route Handler)	This repo is EmailJS’s only caller in the whole system
Push	Firebase Admin SDK (Node.js, service account)	Server-authoritative sends, cannot be spoofed by a client
Hosting	Vercel	Consistent with the website; auto-deploy from <code>main</code>

4. Icons & Visual Assets — Admin-Specific Decision Rule

1. **Any UI glyph a back-office tool needs** (dashboard, chart, filter, sort arrows, checkmark, cross, warning triangle, clock/pending, shield/verification, wallet, megaphone/announcement, user-gear/admin, map pin, phone, envelope, document, eye/view, download, search, trash, edit-pencil)? → **Lucide Icons** (`lucide-react`) — identical library to the public website, for total visual consistency between the two Vercel-hosted Next.js properties.
2. **A brand-consistent empty-state illustration per module** (empty Verification Queue, empty Bookings, empty Disputes, empty Finance Queue) that a plain icon can’t convey warmly enough? → PNG asset via `01_ai_images.json` (\$1.8) — reuse the same flat-vector teal/orange illustration style as the consumer apps’ empty states so the whole system feels like one product family, just note in the prompt that these are for an **internal admin tool**, so a slightly more neutral/professional (less playful) tone than the patient-facing app illustrations is appropriate.
3. **A simple status dot, progress ring, or badge shape?** → Draw directly with Tailwind/inline SVG — don’t spend an AI image slot on it.
4. **Admin avatar placeholders** (5 roles, generic silhouettes, no real faces) → small AI-generated set, or simply a `Lucide UserCircle` icon tinted per role color — either is acceptable here since this is internal-only UI, not a brand

5. Access Control — 5 Admin Roles (authoritative, enforced at the database layer)

Role	Can Access
Super Admin	Everything, including creating/removing admin accounts, editing commission/pricing rules, and the service-role-key-backed override actions
Verification Admin	Verification Queue (A-03/A-04), Assessment oversight (A-05)
Support Admin	Disputes (A-09), CMS modules (A-14–A-20), general support tickets
Finance Admin	Payment Accounts (A-11), Payment Verification Queue (A-12), Payouts (A-13), commission reports (part of A-21)
Emergency Admin	Emergency/SOS module (A-08), incident report review

Enforcement: every table has RLS policies keyed to the JWT’s role claim (set once at admin-account-creation time by a Super Admin, never self-assignable). The UI additionally hides inaccessible sidebar sections per role, but **the real enforcement is server-side** — a Support Admin’s browser cannot successfully query the payment_accounts table even by calling the API directly, because RLS denies it regardless of what the UI shows.

2FA: TOTP-based, mandatory for Super Admin and Finance Admin at minimum (strongly recommended for all five). Every login from a new device/browser triggers an EmailJS alert email (\$1.6 template) to the account owner.

Audit log: every create/update/delete action anywhere in this panel writes a row to admin_audit_log (admin_id, action, table_affected, record_id, before/after snapshot where practical, timestamp). This is non-negotiable given how much of this system (manual payments, manual verification) depends on trustworthy human judgment calls — the audit log is what makes those calls reviewable later.

6. Total Screen Count

23 screens, organized into 9 modules, plus the shared dashboard shell (sidebar + top bar) that wraps all of them. Every screen below inherits the design system from §2 automatically — the tables only call out screen-specific content, components, and connections to the three client apps.

7. Complete Screen-by-Screen Blueprint

7.1 Authentication (1 screen)

ID	Screen	Content & Key Components
A-01	Admin Login	Email + Password (Supabase Auth, admin role only — this login form is entirely separate from the patient/professional login on the other three apps), TOTP 2FA code entry on second step, “new device” email alert fires on success from an unrecognized browser fingerprint

7.2 Dashboard Shell & Overview (1 screen)

ID	Screen	Content & Key Components
A-02	Dashboard Overview	Role-aware sidebar (only shows sections the logged-in admin’s role can access, per §5) + top bar (admin name/avatar, logout). Overview cards: Today’s Bookings, Pending Verifications, Pending Payment Confirmations, Active SOS Alerts (if any — rendered in urgent red if non-zero), Today’s Revenue (real, computed, genuinely PKR 0 until real bookings exist). Recent activity feed

(last 20 admin_audit_log rows). **This screen is a true empty dashboard on first deploy** — every card reads zero/none until real data exists, per the zero-fake-data rule.

7.3 Content Management System — CMS (7 screens)

These modules directly power the Website’s public pages (\$7.1 of the Web Blueprint) and every app’s “About/Support” screens — nothing here is cosmetic, it is live production content.

ID	Screen	Content & Key Components
A-14	FAQ Manager	Table of Q&A pairs (question, answer, category: Patient/Professional, order), Add/Edit modal with a rich-text answer field, drag-to-reorder, Delete with confirmation. Saving calls POST /api/revalidate?path=/faq so the website updates within seconds. Single-record form: support phone, WhatsApp number, support email, live-chat hours, office address, and a repeatable “Emergency Number” list (label + number, e.g. Rescue 1122, Police 15, Edhi 115, Women’s Helpline 1099) — this exact list is what powers the Emergency screen (P-25/PR-equivalent) on all three client apps, so accuracy here is safety-critical.
A-15	Support Center Manager	Three tabs: Terms & Conditions, Privacy Policy, Refund Policy — rich-text editor per page, version history (every save creates a new version row with updated_at, old versions kept, never hard-deleted, for compliance)
A-16	Legal Pages Manager	Mission/vision text fields, founder note, company story, team member list (name, role, photo — Cloudinary upload)
A-17	About Us Manager	List of banners (text, optional image, target audience: All/Patients/Professionals/City-specific, start date, end date), Add/Edit/Delete, toggle active/inactive instantly
A-18	Announcements / Banners Manager	Editable table matching the master pricing structure (Nurse Home Visit, Dedicated Nurse 8/12/24h, Hospital Companion, Navigator, GP Online/Home, Specialist Online/Premium — Budget & Premium columns), plus commission percentage fields (default 10%, 8% for medicine) and surge multiplier fields (peak +20%, emergency +50%) — editing here changes what every app shows immediately (via the same CMS/pricing endpoint), so include a confirmation step before saving.
A-19	Service Pricing Manager	This is the screen that makes the dynamic registration engine work. Table of every professional role (Nursing Division roles, Medical/Dental, Allied Rehab & Diagnostics, Pharmacy Operations, Business Partners, etc.), each row editable: role label, category, required documents list (add/remove document types freely), whether CNIC/affiliation/assessment is required, minimum age. Adding a brand-new professional role here requires zero code changes in any
A-20	Role Requirements Manager	

of the three client apps — they all read this table live at registration time.

7.4 Verification & Assessment (3 screens)

ID	Screen	Content & Key Components
A-03	Verification Queue	Table of pending professional applicants (name, role, submitted date, days waiting). Row click opens a detail panel: every required document (per that role’s A-20 configuration) shown via a signed Cloudinary URL (never a public link), Approve/Reject per document (not just per applicant), mandatory rejection reason field that triggers the doc_rejected EmailJS template automatically.
A-04	Applicant Detail & History	Full applicant profile, document review history (who approved/rejected what and when — pulled from admin_audit_log), “Currently working at” affiliation field shown prominently, action buttons: Approve All & Advance to Assessment / Reject Application / Request Additional Document
A-05	Assessment Oversight	Two tabs: Results (every attempt, every phase, score, pass/fail, badge awarded — filterable by professional/role/date) and Question Bank Management (add/edit/retire questions per phase: Basic Knowledge 30Q pool, Practical Scenario 20Q pool, Ethics 20Q pool — each question has text, options, correct answer, and an active/retired flag so retiring a question doesn’t break historical result records). Manual “Trigger Re-Assessment” action available here for a specific professional (e.g. after a rating drop or complaint).

7.5 Bookings, Live Map & Emergency (3 screens)

ID	Screen	Content & Key Components
A-06	All Bookings	Full searchable/filterable table (TanStack Table): status, date, patient, professional, service type, price, payment status. Row click → full booking detail (read-only for normal cases; Super Admin can force-edit status in a genuine dispute-resolution context, logged to admin_audit_log).
A-07	Live Bookings Map	Real-time map (Leaflet, Firestore bookings_live/presence streams) showing every currently-active visit’s GPS position simultaneously — the one screen in the whole panel that is genuinely “live” the moment real bookings start flowing; empty map with a “No active visits right now” message otherwise.
A-08	Emergency / SOS Alerts	Dedicated Emergency-Admin view. Incoming SOS alerts trigger an audible browser alert (<audio> autoplay on a real-time Firestore listener event) plus a full-screen urgent card: live GPS (Leaflet), one-click “Call Professional” / “Call Patient” (tel: links), and an incident-report form the admin fills in during/after the call, saved to that patient’s access_logs/an

incident_reports table. Resolved incidents move to a searchable history below the live section.

7.6 Finance — Manual Payment System (3 screens)

ID	Screen	Content & Key Components
A-11	Payment Accounts	List of the platform’s receiving accounts (JazzCash number + title, Easypaisa number + title, Bank name + IBAN + title), each with an active/inactive toggle, Add/Edit/Delete — this exact list is what every client app’s Booking Payment screen (P-18 equivalent) renders live.
A-12	Payment Verification Queue	Table of bookings sitting in payment_pending_verification: patient name, amount, method, transaction ID entered by the patient, and the uploaded screenshot via a signed Cloudinary URL. “Confirm Paid” or “Reject/Ask to Resend” buttons, each requiring a short note, each logged. Confirming releases the booking to the professional in real time (Firestore booking_status_events update, visible instantly on whichever app the professional is using).
A-13	Professional Payouts	List of professionals with pending earnings, filterable by requested schedule (Daily/Weekly/Monthly/On-demand), minimum PKR 500 enforced by validation. “Mark Paid” logs the payout with timestamp + admin ID; commission and revenue summary cards on this same screen are auto-calculated from real completed bookings only (genuinely PKR 0 until real bookings exist).

7.7 Reviews Moderation (1 screen)

ID	Screen	Content & Key Components
A-10	Reviews Moderation	Queue of pending reviews and pending professional replies-to-reviews (both are moderated before publishing, per the founder’s core feature spec). Approve / Edit-for-policy-violation / Reject actions, with the two-way rating system visible (patient-rates-professional and professional-rates-patient both flow through here).

7.8 Disputes (1 screen, shared with Support Admin)

ID	Screen	Content & Key Components
A-09	Disputes	Patient-vs-professional dispute queue, evidence attachments (chat excerpts from Firestore chat_threads, photos via signed Cloudinary URLs), resolution notes field, and a “Trigger Refund” action that feeds back into the Finance module (A-12/A-13) as a manual refund task rather than an automated reversal, consistent with the manual-payments philosophy.

7.9 Analytics (1 screen)

ID	Screen	Content & Key Components
----	--------	--------------------------

A-21	Analytics Dashboard	Recharts-powered: bookings over time, revenue by service category, top-rated professionals, city-wise demand, referral leaderboard snapshot. Every chart is wired to a real query from day one and simply renders an empty/zero state gracefully — never sample/demo data during development or after launch.
------	---------------------	--

7.10 Admin User Management & Notifications (2 screens)

ID	Screen	Content & Key Components
A-22	Admin User Management	List of admin accounts (Super Admin only can access this screen), Add New Admin (assign one of the 5 roles from §5), Deactivate/Reactivate, force-2FA-reset. Full <code>admin_audit_log</code> viewer lives here too — searchable by admin, action type, date range. Compose a push notification (title, body, priority tier: mandatory/important/optional per the apps' 3-tier system), target audience (All / Patients / Professionals / specific city / specific role), schedule now or later. Sends via the Firestore Admin SDK server-side — the agent should confirm delivery lands on a real Android device, a real iPhone, and a real browser (web push) before considering this screen complete, per the §1.9 connection-verification requirement.
A-23	Notifications Composer	

8. Navigation Map

A-01 Login (+ 2FA) → A-02 Dashboard Overview
Sidebar (role-filtered, always from A-02): - Verification: A-03 Queue → A-04 Applicant Detail → A-05 Assessment Oversight - Bookings: A-06 All Bookings → A-07 Live Map - Emergency: A-08 SOS Alerts - Disputes: A-09 - Reviews: A-10 - Finance: A-11 Payment Accounts → A-12 Verification Queue → A-13 Payouts - Content (CMS): A-14 FAQ — A-15 Support — A-16 Legal — A-17 About — A-18 Announcements — A-19 Pricing — A-20 Role Requirements - Analytics: A-21 - Admin: A-22 Admin Users & Audit Log — A-23 Notifications Composer

Implement as one authenticated route group (`app/(dashboard)/`) with a persistent sidebar reading the logged-in admin's role claim once at layout-mount to decide which links render — re-verify the role server-side on every route handler too, never trust the client-rendered sidebar as the actual security boundary (§5).

9. Cross-Platform Data Sync — The Complete Picture

This table is the single reference showing **how every other repository connects through this one**, and should be treated as the definitive answer to “is everything really connected in real time?”

Data / Action	Written By	Read By (real-time where noted)
New patient/professional account	Android, iOS, or Web app (Firestore Auth + Supabase <code>users/patients/professionals</code> insert)	This Admin Panel (A-03 Verification Queue, A-06 Bookings, A-22) sees it immediately via Supabase read
Document upload	Whichever app the professional used (Cloudinary signed upload + Supabase <code>documents</code> row)	A-03/A-04, via signed Cloudinary URL requested from this repo
Verification approve/reject	This Admin Panel (A-03/A-04)	The professional's own app (Android/iOS/Web — whichever they're using) via a Supabase real-time subscription or Firestore

		verification_status doc — must update without a manual refresh
New booking, booking status changes	Patient's app (any of the three) writes to Supabase bookings + Firestore bookings_live/booking_status_events	A-06 (table), A-07 (live map) here; also the professional's app in real time
SOS trigger	Patient's app (any of the three), writes an emergency event	A-08 here, must sound an audible alert within seconds
Payment submitted (transaction ID + screenshot)	Patient's app, any platform	A-12 here
Payment confirmed	This Admin Panel (A-12)	The patient's app (booking unlocks) and the professional's app (booking now shows as paid), both in real time
Payout marked paid	This Admin Panel (A-13)	The professional's app (Earnings screen updates)
Review submitted	Patient's or professional's app	A-10 here (moderation queue)
Review approved	This Admin Panel (A-10)	Now visible on the professional's public profile across all three client apps
FAQ/Support/Legal/About/Pricing/Role-Requirements edits	This Admin Panel (A-14-A-20)	The Website's public pages (via GET /api/cms/* + the /api/revalidate webhook) and all three apps' dynamic registration engine / About-Support screens
Push notification	This Admin Panel (A-23, Firebase Admin SDK)	All three client apps' notification feed + native OS-level push
Every admin action	This Admin Panel , any screen	admin_audit_log, viewable only from A-22

If any row in this table doesn't work end-to-end on a real device/browser test, the corresponding feature is **not done**, regardless of how complete the UI looks in isolation — this mirrors the §1.9 verification requirement.

10. Build & Deployment Pipeline

1. Every push to main auto-deploys on **Vercel**; pull requests get preview URLs pointed at the homigo-care-dev Firebase/Supabase environment so admin-side testing never touches production data (protecting the zero-fake-data rule on the live system).
2. Because this repo holds the most sensitive secrets in the entire project (Supabase service-role key, Cloudinary API secret, EmailJS private key, Firebase service account), restrict who can merge to main and enable Vercel's deployment protection (password or SSO-gated preview access) if available on the plan in use.
3. After any schema change to role_requirements, cms_content, or payment_accounts, trigger /api/revalidate for the Website and confirm the three apps pick up the change on their next fetch cycle (they poll/cache per their own blueprints' revalidation windows).

11. Notes for the AI Developer (Guide, Not a Separate File Yet)

This section is a short internal guide for whichever AI agent (Manus AI or otherwise) implements this repository — the founder has indicated a **separate, final “AI working-guidelines” file** will be produced in the next chat to formalize exactly how the agent should behave across all four repositories. Until then, treat these as binding for this repo specifically:

1. **Never fabricate data to make a screen “look complete.”** An empty Verification Queue, an empty Live Map, a PKR 0 revenue card are all correct, expected states on a fresh deploy — build every screen to render them gracefully rather than quietly seeding a demo row to make a screenshot look nicer.
2. **Treat this repo as the source of truth for OTP and CMS content** — if the Android, iOS, or Web blueprint documents describe a behavior that depends on ADMIN_API_BASE_URL, that endpoint must actually exist and work in this repo before the corresponding client-side feature can be considered done.
3. **Every manual, human-judgment action (document approval, payment confirmation, payout, dispute resolution) must be logged** to admin_audit_log — this is not optional polish, it is the accountability mechanism the entire manual-operations model depends on.
4. **Ask before inventing a business rule.** If an edge case isn't covered by the pricing/commission rules already documented (e.g. what happens to commission on a partially-refunded booking), pause and ask the founder rather than silently deciding.
5. **Confirm real connectivity (§1.9) before declaring any phase complete** — a screen that only works against mocked data is not a finished screen in this project.

12. Final QA Checklist Before Calling the Build “Done”

- ☐ All 23 screens in §7 exist and are reachable per the role-filtered sidebar in §5/§8.
- ☐ Every color/spacing value traces back to the shared design tokens in §2 — no inline hex, no arbitrary spacing.
- ☐ Every icon is a Lucide icon, an inline SVG, or a founder-approved AI asset from §1.8 — no emoji icons.
- ☐ 2FA is enforced for Super Admin and Finance Admin at minimum; RLS genuinely blocks out-of-role data access (tested by attempting a direct API call as a lower-privileged role, not just by checking the hidden UI).
- ☐ `admin_audit_log` captures every create/update/delete action across all modules.
- ☐ A-14-A-20 (CMS) changes are confirmed to actually update the live Website within the expected revalidation window.
- ☐ A-03/A-04 approval/rejection is confirmed to update a real professional’s app (any of the three) in real time.
- ☐ A-12 payment confirmation is confirmed to unlock a real booking on a real patient’s app in real time.
- ☐ A-08 SOS alert is confirmed to trigger an audible alert here within seconds of a real trigger from a client app.
- ☐ A-23 push notification is confirmed to arrive on a real Android device, a real iPhone, and a real browser.
- ☐ No table anywhere in this panel (or its connected client apps) contains seed/demo data — every empty state is genuine.
- ☐ `public/assets_pending/01_ai_images.json` is valid JSON, 20–50 entries, every `destination_path` used somewhere real.
- ☐ This repo’s `/api/otp/*` and `/api/cms/*` endpoints are live and are the actual endpoints the Android, iOS, and Web repos are configured to call (`ADMIN_API_BASE_URL` matches across all four repos).
- ☐ Site is live on Vercel at the intended admin domain with auto-deploy from `main` confirmed working.